

merci-audit:silence-style

Automatización Extendida: Compilador Data-Driven para la Biblioteca

Art de Côté | Cuadernillo

Al diseñar la arquitectura de nuestro CMS Headless en mercedev.es, nos enfrentamos a un problema recurrente: el Agente Autónomo del glosario extraía e inyectaba nuevas definiciones técnicas directamente en el archivo Markdown (`glosario-tecnico.md`). Sin embargo, esto planteaba dos desafíos significativos:

1. **Fragilidad de Formato:** A pesar de los Prompts estrictos utilizados por la IA, ocasionalmente se producían errores visuales como `**Término**` en lugar de `### Término` . Un simple error tipográfico podía romper la visualización del documento.
2. **Desalineación Filosófica:** El glosario Markdown estaba siendo tratado como una base de datos cuando, en realidad, en nuestro ecosistema SSG (Static Site Generator), la *Única Fuente de Verdad* es la bitácora de documentación. El glosario no debe ser un documento de trabajo, sino un *Art de Côté* (artefacto compilado).

Para resolver estos problemas, decidimos refactorizar el script `merci-glosario.py` , abandonando la inyección directa en Markdown y adoptando un enfoque **Data-Driven (Basado en JSON)**. Este cambio nos permitió:

1. **Estado JSON Maestro:** Extrajimos el histórico completo del glosario hacia un nuevo archivo `glosario-tecnico.json` . Este JSON almacena los términos, definiciones, el registro de archivos origen y una *lista negra* de términos descartados.
2. **IA Determinista:** Configuramos el cliente HTTP para requerir explícitamente `format: "json"` a la API de Ollama. La IA ahora no devuelve texto libre, sino un objeto estructurado predecible e imposible de romper visualmente.
3. **Compilación Dinámica (Build-Time):** El script ahora opera como un compilador, sincronizando los nuevos términos con el JSON y generando y sobrescribiendo el archivo `glosario-tecnico.md` desde cero cada vez que se ejecuta el pipeline (`merci-total.py`). Esto garantiza que el artefacto resultante esté matemáticamente ordenado (alfabéticamente) y formateado de forma impecable.

La refactorización del enfoque mostró varios beneficios significativos:

- **JSON sobre Markdown para Agentes:** Cuando un Agente Autónomo tiene permisos de escritura continua, el formato de almacenamiento de estado debe ser estricto (JSON o Base de Datos). Dejar que un LLM

manipule código Markdown interactivo o en bruto introduce Deuda Técnica impredecible.

- **Aislamiento del Origen:** Consolidar el paradigma de que "los archivos .md finales son de solo lectura (compilados)" simplifica el mantenimiento. Si el Markdown se corrompe por acción humana, la próxima ejecución del Build lo restaurará instantáneamente desde el JSON.

Antes, nuestra Inteligencia Artificial escribía el diccionario técnico directamente sobre la página final, causando errores visuales si la IA se equivocaba con el formato o la puntuación. Para solucionarlo, ahora la IA guarda las definiciones en un "cajón de datos" estructurado e invisible. Luego, nuestro sistema actúa como una imprenta mecánica: coge esos datos seguros y fabrica la página perfecta y ordenada alfabéticamente desde cero cada vez. Así nos aseguramos de que el resultado nunca se rompa visualmente, por mucho que la IA se equivoque.

Para obtener más detalles sobre esta evolución en nuestro proceso de automatización, te invitamos a leer el [cuadernillo completo](#).